

SIMD&SIMT新同构编程： Regbase 编程进阶 & 调优工具FlashVF实操演练

作者：谢盛伟、芦葳、戴伟

时间：2026/6

目录

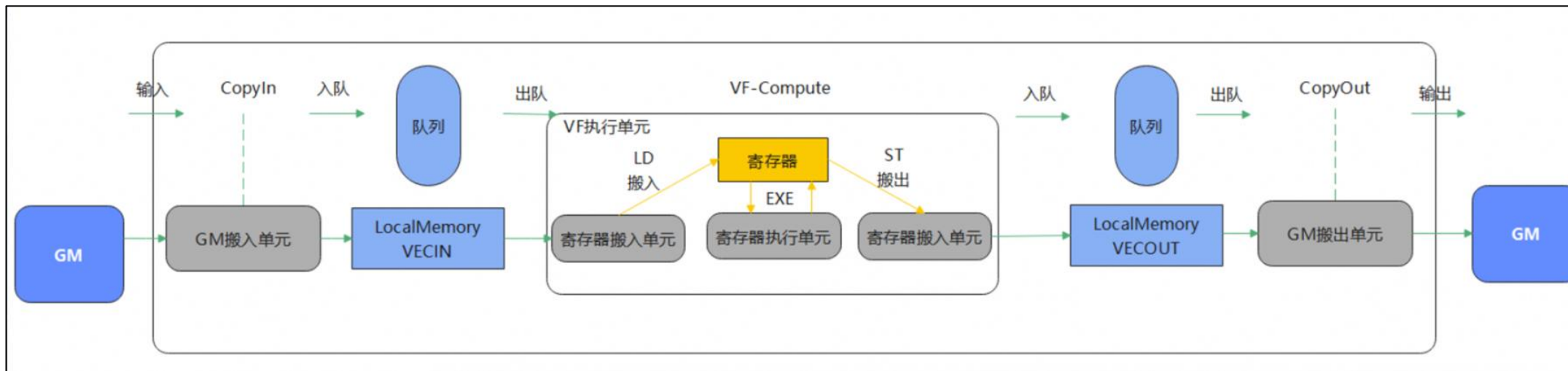
Part 1 Regbase编程回顾

Part 2 Regbase编程高性能实践

Part 3 基于FlashVF的性能优化实践

Regbase编程回顾

往期回顾: [基于下一代硬件的Ascend C Reg矢量编程](#)



Membase编程:

- 编程处理的数据对象: LocalMemory上的Tensor (LocalTensor)
- 数据流: GlobalTensor -> LocalTensor -> **Membase计算** -> LocalTensor->GlobalTensor

Regbase编程:

- 编程处理的数据对象: 寄存器/RegTensor
- 数据流: GlobalTensor -> LocalTensor -> **RegTensor**-> **Regbase计算** -> **RegTensor**-> LocalTensor->GlobalTensor

减少从UB->寄存器的重复搬运, 提升计算效率

Regbase编程回顾

Membase编程:

```
// ComputeMemBase实现使用的是ascendc提供的基础api, 开发者不感知寄存器。可认为数据在UB上完成vector计算
__aicore__ inline void ComputeMemBase(int64_t i0Extent, int64_t ubLoopIdx)
{
    // 使用Ascendc提供的队列管理内存
    bufferIn0_ = queIn0_.DeQueue<float>();
    bufferOut0_ = queOut0_.AllocTensor<float>();
    // 具体计算逻辑
    MulS(bufferOut0_, bufferIn0_, -1.0, i0Extent); // i0Extent 根据实际tiling切分大小设置
    Exp(bufferOut0_, bufferOut0_, i0Extent);
    AddS(bufferOut0_, bufferOut0_, scalarOne, i0Extent);
    Div(bufferOut0_, bufferOut0_, bufferIn0_, i0Extent);
    // 释放UB空间, 并入队 (使用ascendc的队列管理内存)
    queIn0_.FreeTensor(bufferIn0_);
    queOut0_.EnQueue<float>(bufferOut0_);
}
```

→ 输入LocalTensor

→ Membase计算指令

→ 输出LocalTensor

Regbase编程: 基于simd_vf承载

```
// RegBase和MemBase编程的主要差异是compute函数的实现;
// ComputeRegBase实现使用的是微指令, 现将数据从UB上load到寄存器, 在寄存器上进行相应的vector计算,
// 计算完之后, 再将数据store到UB
__simd_vf__ inline void ComputeRegBase(int64_t i0Extent, int64_t ubLoopIdx,
__ubuf__ float *bufferIn0_, __ubuf__ float *bufferOut0_)
{
    RegTensor<float> vreg0, vreg1, vreg2, vreg3, vreg4;
    MaskReg preg0;
    uint32_t size = i0Extent;
    uint16_t vloopNum = (i0Extent + (AscendC::VECTOR_REG_WIDTH / sizeof(float)) - 1) /
        (AscendC::VECTOR_REG_WIDTH / sizeof(float));
    __ubuf__ float *bufferIn0Addr = (__ubuf__ float *)bufferIn0_.GetPhyAddr();
    __ubuf__ float *bufferOut0Addr = (__ubuf__ float *)bufferOut0_.GetPhyAddr();
    for (uint16_t i = 0; i < vloopNum; i++) {
        preg0 = AscendC::Reg::UpdateMask<float>(size);
        AscendC::Reg::DataCopy<float, AscendC::Reg::LoadDist::DIST_NORM>(vreg0,
            bufferIn0Addr + i * (AscendC::VECTOR_REG_WIDTH / sizeof(float)));
        AscendC::Reg::MulS<float, float, AscendC::Reg::MaskMergeMode::ZEROING>(vreg1, vreg0,
            static_cast<float>(-1.0) * tilingDataPtr->scale, preg0);
        AscendC::Reg::Exp<float, AscendC::Reg::MaskMergeMode::ZEROING>(vreg2, vreg1, preg0);
        AscendC::Reg::AddS<float, float, AscendC::Reg::MaskMergeMode::ZEROING>(vreg3, vreg2,
            static_cast<float>(1.0), preg0);
        AscendC::Reg::Div<float, AscendC::Reg::MaskMergeMode::ZEROING>(vreg4, vreg3, preg0);
        AscendC::Reg::DataCopy<float, AscendC::Reg::StoreDist::DIST_NORM_B32>(
            bufferOut0Addr + i * (AscendC::VECTOR_REG_WIDTH / sizeof(float)), vreg4, preg0);
    }
}

__aicore__ inline void Compute(int64_t i0Extent, int64_t ubLoopIdx)
{
    bufferIn0_ = queIn0_.DeQueue<float>();
    bufferOut0_ = queOut0_.AllocTensor<float>();
    asc_vf_call<ComputeRegBase>(i0Extent, ubLoopIdx, bufferIn0_, bufferOut0_);
    queIn0_.FreeTensor(bufferIn0_);
    queOut0_.EnQueue<float>(bufferOut0_);
}
```

→ 申请寄存器: 向量寄存器 (RegTensor) 和掩码寄存器 (MaskReg)

→ 输入/输出LocalTensor对应的ub地址

→ LocalTensor->寄存器的搬入指令

→ 寄存器间的计算指令

→ 寄存器->LocalTensor的搬出指令

→ 调用simd_vf, 相当于把vector运算封装到一个超大Membase指令里

算子性能分析工具：算子流水图

图源：cannsim仿真report中的json文件，使用浏览器内置tracing工具打开



- 执行单元划分（从上到下）：
- 01_SCALAR: 标量操作
 - 02_SCALARLDST: 标量加载
 - 04_MTE2: 数据从HBM/L2搬运到UB
 - 07_MTE3: 数据从UB搬运到HBM/L2
 - 10_PUSHQ: 指令进队列
 - 11_RVECSU: vec指令进队列
 - 12_RVECEX: vec指令执行
 - 13_RVECLD: vec指令寄存器加载
 - 14_RVECST: vec指令寄存器加载

目录

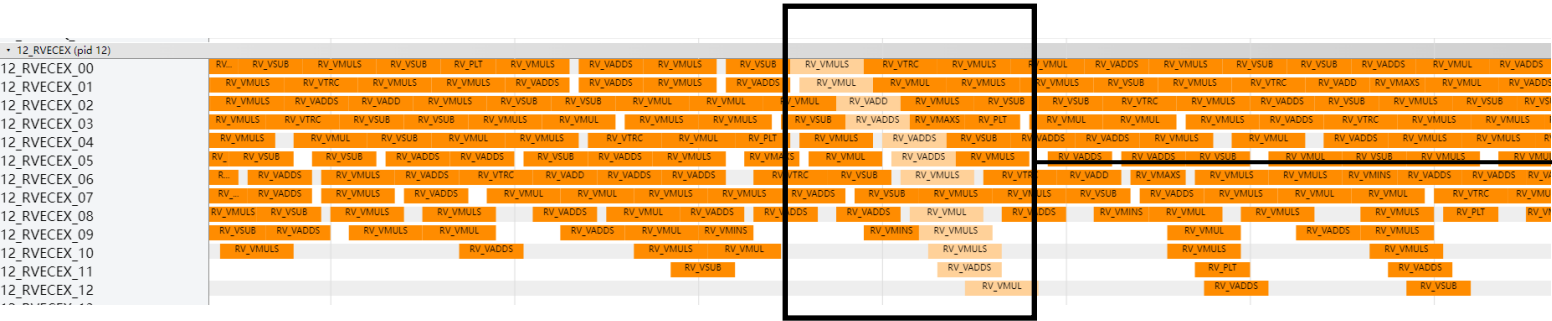
Part 1 Regbase编程回顾

Part 2 Regbase编程高性能实践

Part 3 基于FlashVF的性能优化实践

前置知识点：时间并行和乱序执行

时间并行（流水线执行）：对于一组没有数据依赖的指令，可以通过流水线实现时间并行



单条流水并行度达到12

=> 一条指令未执行完成时就可以继续执行下一条没有数据依赖的指令

乱序执行：在这种范式中，处理器根据输入数据的可用性确定执行指令的顺序，而非原始代码顺序

这种范式通过以下步骤挑选可执行的指令先执行：

1. 指令获取。
2. 指令被发送到一个指令序列中（也称执行缓冲区或者保留站）。
3. 指令将在序列中等待直到它的数据运算对象是可以获取的。然后指令被允许在先进入的（较旧的）指令之前离开序列缓冲区。
4. 指令被分配给一个合适的功能单元并由之执行。
5. 结果被放到一个序列中。
6. 仅当所有在该指令之前的指令都将他们的结果写入寄存器后，这条指令的结果才会被写入寄存器中。这个过程被称为毕业或者退休周期。

前置知识点：循环Unroll

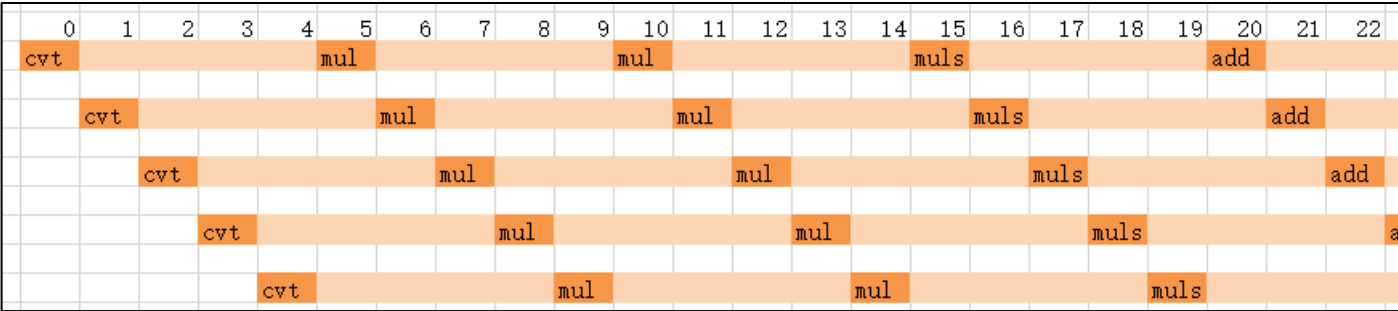
循环Unroll：硬件进行循环unroll再乱序执行，无数据依赖的循环迭代间指令可以并行

原始指令流：循环体内顺序依赖

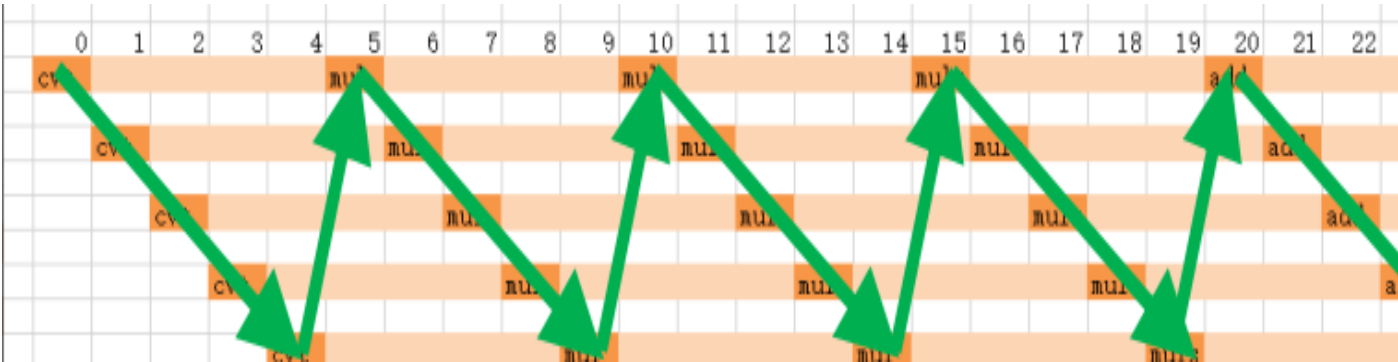
```
for(.....)
  x1 = cvt(x0)
  x2 = mul(x1,...)
  x3 = mul(x2,...)
  x4 = muls(x3,...)
  x5 = add(x4,...)
```



展开后指令流：五次循环的指令流被展开到同一执行平面内

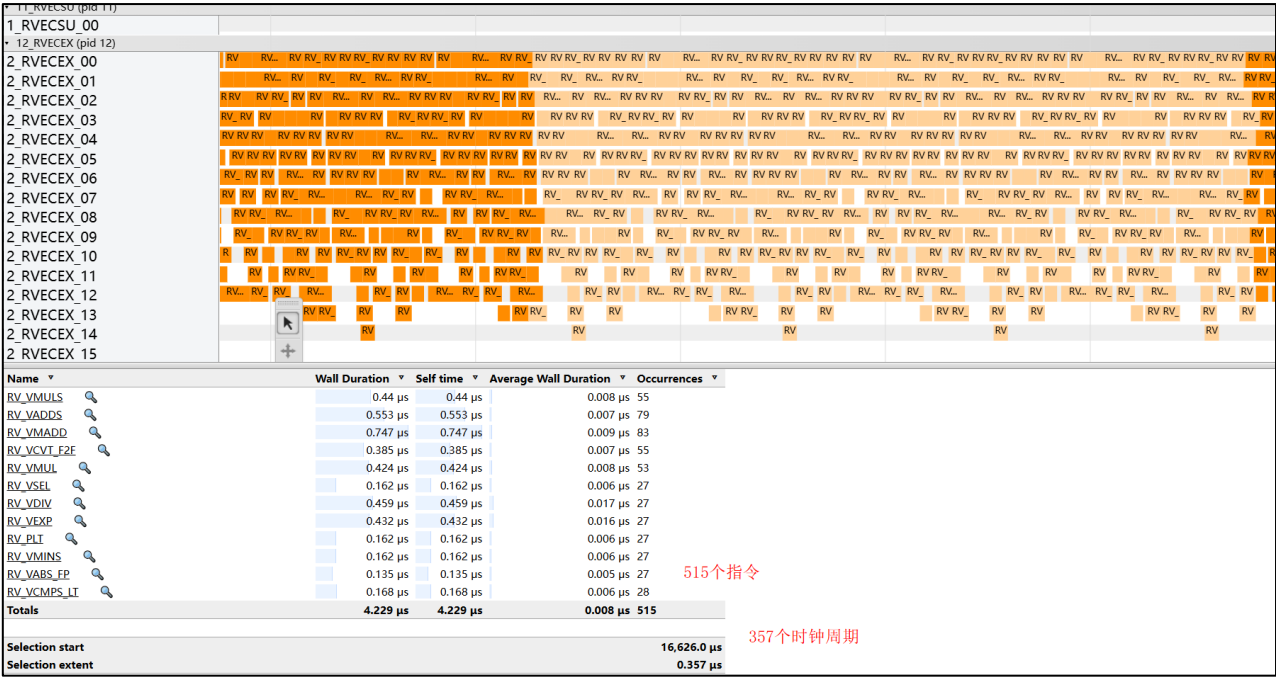


实际执行顺序：先执行五次循环的cvt，再执行五次循环的mul，以此类推



VF性能指标：IPC

IPC（每周期指令，instructions per cycle， 又称 instructions per clock）是衡量处理器性能的一个指标： 每个时钟周期执行的平均指令数。



在Tracing中打开流水图，框选中一段时间内RVECEX的全部微指令

- Totals:515 => **515条指令**
- Selection extent: 0.357=> **357个时钟周期**
- $IPC=515/357=1.44$

IPC理论上限是2.0，一般达到**1.4**会是性能较高的实现

措施一：VF融合，减少重复搬运和头开销

融合前：计算指令占比较少，ub和寄存器间的通信损耗较大，性能较低

```
Python
1  _VEC_SCOPE__ {
2      for (uint16_t loopIdx = 0; loopIdx < loopNum; loopIdx++) {
3          mask = MicroAPI::UpdateMask<T, MicroAPI::RegTraitNumOne>(count);
4          // OpCopyIn
5          MicroAPI::DataCopy(vregInput1, (__ubuf__ T*)(src1Addr + loopIdx * v1Size));
6          MicroAPI::DataCopy(vregInput2, (__ubuf__ T*)(src2Addr + loopIdx * v1Size));
7
8          MicroAPI::Add(vregOutput, vregInput1, vregInput2, mask);
9          // OpCopyOut
10         MicroAPI::DataCopy((__ubuf__ T*)(dstAddr + loopIdx * v1Size), vregOutput, mask);
11     }
12 }
13
14 _VEC_SCOPE__ {
15     for (uint16_t loopIdx = 0; loopIdx < loopNum; loopIdx++) {
16         mask = MicroAPI::UpdateMask<T, MicroAPI::RegTraitNumOne>(count);
17         // OpCopyIn
18         MicroAPI::DataCopy(vregInput1, (__ubuf__ T*)(dstAddr + loopIdx * v1Size));
19         MicroAPI::DataCopy(vregInput2, (__ubuf__ T*)(src3Addr + loopIdx * v1Size));
20
21         MicroAPI::Add(vregOutput, vregInput1, vregInput2, mask);
22         // OpCopyOut
23         MicroAPI::DataCopy((__ubuf__ T*)(dstAddr + loopIdx * v1Size), vregOutput, mask);
24     }
25 }
```

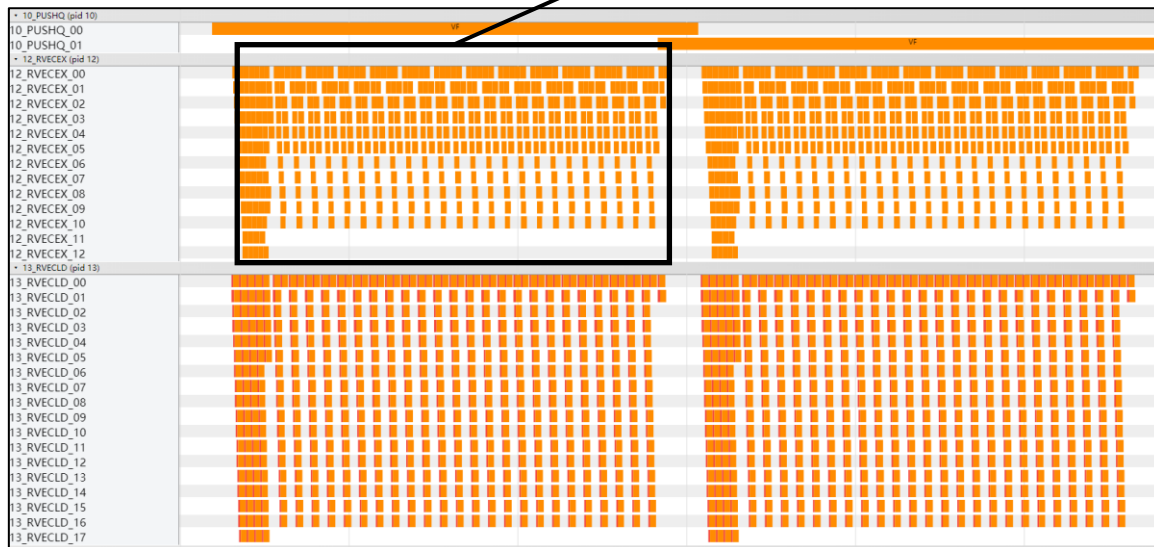


融合后：计算指令占比较高，通信损耗相对较低，性能较好

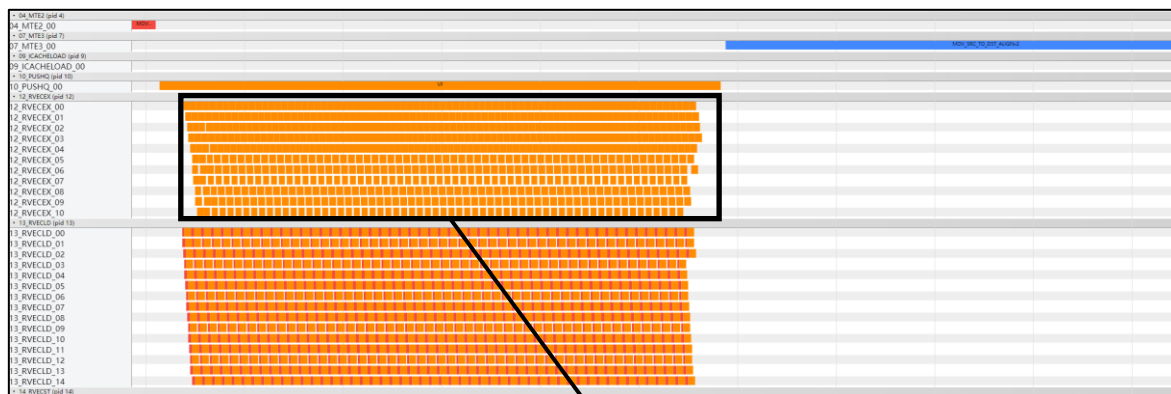
```
Python
1  _VEC_SCOPE__ {
2      for (uint16_t loopIdx = 0; loopIdx < loopNum; loopIdx++) {
3          mask = MicroAPI::UpdateMask<T, MicroAPI::RegTraitNumOne>(count);
4          // OpCopyIn
5          MicroAPI::DataCopy(vregInput1, (__ubuf__ T*)(src1Addr + loopIdx * v1Size));
6          MicroAPI::DataCopy(vregInput2, (__ubuf__ T*)(src2Addr + loopIdx * v1Size));
7          MicroAPI::DataCopy(vregInput3, (__ubuf__ T*)(src2Addr + loopIdx * v1Size));
8
9          MicroAPI::Add(vregOutput, vregInput1, vregInput2, mask);
10         MicroAPI::Add(vregOutput, vregOutput, vregInput3, mask);
11         // OpCopyOut
12         MicroAPI::DataCopy((__ubuf__ T*)(dstAddr + loopIdx * v1Size), vregOutput, mask);
13     }
14 }
```



指令稀疏，IPC较低



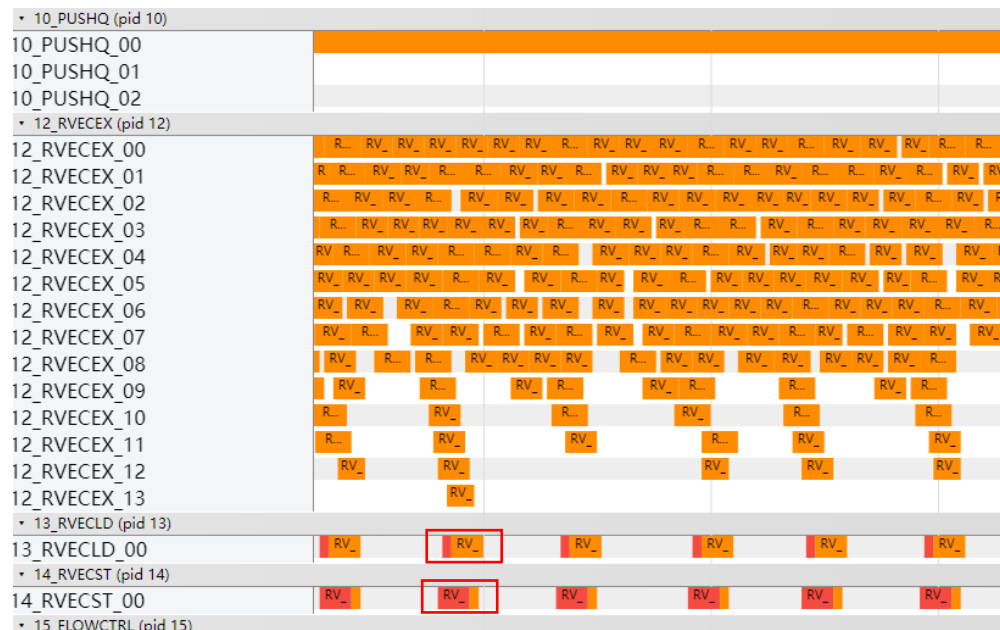
指令稠密，IPC较高



措施二：对于复杂数学计算VF，通过拆分减少资源溢出

基于多项式逼近的COS代码：循环体内30+指令，达到队列上限

```
1 template <typename T>
2 _alcore_ inline void CosPolynomial(_ubuf_ T *dst, _ubuf_ T *src, uint32_t calCount, uint16_t repeat)
3 {
4     _VEC_SCOPE_
5     {
6         MicroAPI::RegTensor<float> x;
7         MicroAPI::RegTensor<float> round;
8         MicroAPI::RegTensor<float> kpl;
9         MicroAPI::RegTensor<float> srcReg;
10        MicroAPI::RegTensor<float> dstReg;
11        constexpr uint32_t oneRepSize = GetVecLen() / sizeof(float);
12        MicroAPI::MaskReg mask;
13        MicroAPI::MaskReg maskAll = MicroAPI::CreateMask<uint8_t>();
14        for (uint16_t i = 0; i < repeat; i++) {
15            mask = MicroAPI::UpdateMask<float>(calCount);
16            MicroAPI::DataCopy(srcReg, src + i * oneRepSize);
17            // x = round * k * pi / 2
18            MicroAPI::Muls(round, srcReg, COS_PI_FOR_X_TODIV, mask);
19            MicroAPI::Adds(round, round, COS_POINT_FIVE, mask);
20            // tie to even
21            MicroAPI::Truncate<float, RoundMode::CAST_ROUND, MicroAPI::MaskMergeMode::ZEROING>(round, round, mask);
22            // x = k * pi / 8
23            MicroAPI::Muls(kpl, round, PI_8, mask);
24            MicroAPI::Sub(x, srcReg, kpl, mask);
25            // x = k * pi / 4
26            MicroAPI::Muls(kpl, round, COS_KPI_FIRS_PI_MUL5, mask);
27            MicroAPI::Sub(x, x, kpl, mask);
28            // x = x + COS.PI_DOWN
29            MicroAPI::Adds(x, x, COS_PI_DOWN, mask);
30            // x = k * pi / 2
31            MicroAPI::Muls(kpl, round, COS_KPI_TMT_PI_MUL5, mask);
32            MicroAPI::Sub(x, x, kpl, mask);
33            // x = k * pi / 8
34            MicroAPI::Muls(kpl, round, COS_KPI_THIR_PI_MUL5, mask);
35            MicroAPI::Sub(x, x, kpl, mask);
36            // x = k * pi / 4
37            MicroAPI::Muls(kpl, round, COS_KPI_FOR_PI_MUL5, mask);
38            MicroAPI::Sub(x, x, kpl, mask);
39            // x = x + COS.PI_RESDOWN_ADDS_NEG
40            MicroAPI::Adds(x, x, COS_PI_RESDOWN_ADDS_NEG, mask);
41            // x*2 = mul(input_x, input_x)
42            MicroAPI::Mul(kpl, x, x, mask);
43            // kover2
44            MicroAPI::Muls(dstReg, round, COS_POINT_FIVE, mask);
45            MicroAPI::Truncate<float, RoundMode::CAST_FLOOR, MicroAPI::MaskMergeMode::ZEROING>(dstReg, dstReg, mask);
46            // kover2floord
47            MicroAPI::Muls(dstReg, dstReg, COS_M4_SCA, mask);
48            // k2
49            MicroAPI::Muls(round, round, COS_K2_SCA, mask);
50            // sign
51            MicroAPI::Add(dstReg, dstReg, round, mask);
52            MicroAPI::Adds(dstReg, dstReg, dstReg, SCALAR_ONE, mask);
53            // res_up = mul(x*2, 2.6892689268926893e-5)
54            MicroAPI::Muls(round, kpl, COS_RES_MULT_SCA, mask);
55            MicroAPI::Adds(round, round, COS_RES_ADDICT_UP, mask);
56            // res_up = mul(res_up, x*2)
57            MicroAPI::Mul(round, round, kpl, mask);
58            MicroAPI::Adds(round, round, COS_2ADDS, mask);
59            // res_up = mul(res_up, x*2)
60            MicroAPI::Mul(round, round, kpl, mask);
61            MicroAPI::Adds(round, round, COS_3ADDS, mask);
62            // res_up = mul(res_up, x*2)
63            MicroAPI::Mul(round, round, kpl, mask);
64            MicroAPI::Adds(round, round, SCALAR_ONE, mask);
65            // sin(x) = xp(x)
66            MicroAPI::Mul(round, round, x, mask);
67            MicroAPI::Mul(dstReg, round, dstReg, mask);
68            MicroAPI::Mins(dstReg, dstReg, SCALAR_ONE, mask);
69            MicroAPI::Maxs(dstReg, dstReg, -SCALAR_ONE, mask);
70            MicroAPI::DataCopy(dst + i * oneRepSize, dstReg, mask);
71        }
72    }
73 }
```



- IPC 1.22，性能有提升空间，需要分析性能可提升的点。
- 代码里，一个循环体内只有一次拷入，一次拷出。观察流水图，图内拷贝指令未并发执行，证明循环是顺序执行的，未unroll。
- 循环未unroll，说明未充分利用处理器乱序执行的能力，存在提升空间。

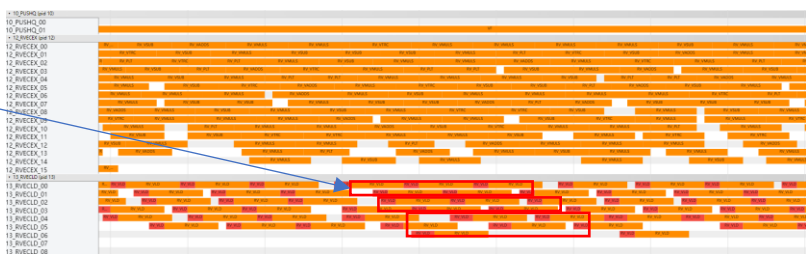
措施二：对于复杂数学计算VF，通过拆分减少资源溢出

VF拆分为三段后，2~3个循环体同步执行，指令并行度高

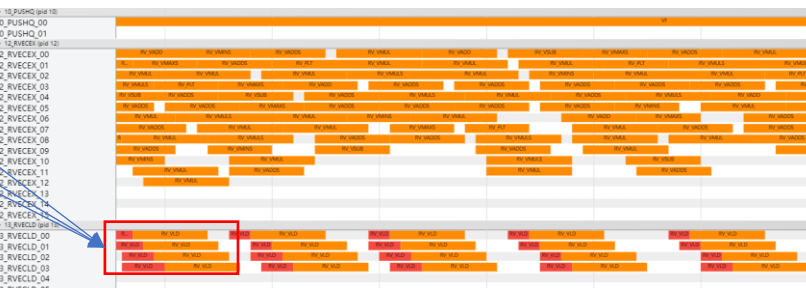
```
1 template<typename T>
2 _alcore_ inline void CosPolynomialAfterFlashVF(_ubuf_T *dst, _ubuf_T *src, uint32_t calCount, uint16_t repeat,
3 _local_mem_float *buffer1000, _local_mem_float *buffer1001, _local_mem_float *buffer1002,
4 _local_mem_float *buffer1003, _local_mem_float *buffer1004, _local_mem_float *buffer1005,
5 _local_mem_float *buffer1006)
6 {
7     __VEC_SCOPE__
8     {
9         RegTensor<float> srcReg, round, kpi, xTmp, dstReg;
10         MaskReg mask;
11         for (uint16_t i = 0; i < repeat; i++) {
12             mask = MicroAPI::UpdateMask<float>(calCount);
13             MicroAPI::DataCopy(srcReg, (_local_mem_float *)src + i * GetVecLen() / sizeof(float));
14             MicroAPI::Mul2(round, srcReg, COS_PI_FIR_X_100IV, mask);
15             MicroAPI::Add2(round, round, COS_POINT_FIVE, mask);
16             MicroAPI::Truncate(float, RoundMode::CAST_ROUND, MicroAPI::MaskMergeMode::ZEROING(round, round, mask));
17             DataCopy((_local_mem_float *)buffer1000 + i * 64, round, mask);
18             MicroAPI::Mul2(kpi, round, PI_0, mask);
19             MicroAPI::Sub(xTmp, srcReg, kpi, mask);
20             MicroAPI::Mul2(kpi, round, COS_KPI_FIR_PI_MUL5, mask);
21             MicroAPI::Sub(xTmp, xTmp, kpi, mask);
22             DataCopy((_local_mem_float *)buffer1001 + i * 64, xTmp, mask);
23             MicroAPI::Mul2(dstReg, round, COS_POINT_FIVE, mask);
24             DataCopy((_local_mem_float *)buffer1002 + i * 64, dstReg, mask);
25         }
26     }
27
28     __VEC_SCOPE__
29     {
30         RegTensor<float> round, xTmp, kpi, dstReg;
31         MaskReg mask;
32         for (uint16_t i = 0; i < repeat; i++) {
33             mask = MicroAPI::UpdateMask<float>(calCount);
34             DataCopy(round, (_local_mem_float *)buffer1000 + i * 64);
35             DataCopy(xTmp, (_local_mem_float *)buffer1001 + i * 64);
36             MicroAPI::Add2(xTmp, xTmp, COS_PI_DOWN, mask);
37             MicroAPI::Mul2(kpi, round, COS_KPI_TW_PI_MUL5, mask);
38             MicroAPI::Sub(xTmp, xTmp, kpi, mask);
39             MicroAPI::Mul2(kpi, round, COS_KPI_THIR_PI_MUL5, mask);
40             MicroAPI::Sub(xTmp, xTmp, kpi, mask);
41             DataCopy((_local_mem_float *)buffer1003 + i * 64, xTmp, mask);
42             MicroAPI::Mul2(kpi, round, COS_KPI_FIR_PI_MUL5, mask);
43             DataCopy((_local_mem_float *)buffer1004 + i * 64, kpi, mask);
44             DataCopy(dstReg, (_local_mem_float *)buffer1002 + i * 64);
45             MicroAPI::Truncate(float, RoundMode::CAST_ROUND, MicroAPI::MaskMergeMode::ZEROING(dstReg, dstReg, mask));
46             DataCopy((_local_mem_float *)buffer1005 + i * 64, dstReg, mask);
47             MicroAPI::Mul2(round, round, COS_K2_SCA, mask);
48             DataCopy((_local_mem_float *)buffer1006 + i * 64, round, mask);
49         }
50     }
51
52     __VEC_SCOPE__
53     {
54         RegTensor<float> round, xTmp, kpi, dstReg;
55         MaskReg mask;
56         for (uint16_t i = 0; i < repeat; i++) {
57             mask = MicroAPI::UpdateMask<float>(calCount);
58             DataCopy(kpi, (_local_mem_float *)buffer1004 + i * 64);
59             DataCopy(xTmp, (_local_mem_float *)buffer1001 + i * 64);
60             MicroAPI::Sub(xTmp, xTmp, kpi, mask);
61             MicroAPI::Add2(xTmp, xTmp, COS_PI_RESDOWN_ADDS_NEG, mask);
62             MicroAPI::Mul(kpi, xTmp, xTmp, mask);
63             DataCopy(dstReg, (_local_mem_float *)buffer1005 + i * 64);
64             MicroAPI::Mul2(dstReg, dstReg, COS_M4_SCA, mask);
65             DataCopy(round, (_local_mem_float *)buffer1006 + i * 64);
66             MicroAPI::Add2(dstReg, dstReg, round, mask);
67             MicroAPI::Add2(dstReg, dstReg, (float)1.0, mask);
68             MicroAPI::Mul2(round, kpi, COS_RES_MULT_SCA, mask);
69             MicroAPI::Add2(round, round, COS_RES_ADD0CT_UP, mask);
70             MicroAPI::Mul(round, round, kpi, mask);
71             MicroAPI::Add2(round, round, COS_2ADDS, mask);
72             MicroAPI::Mul(round, round, kpi, mask);
73             MicroAPI::Add2(round, round, COS_3ADDS, mask);
74             MicroAPI::Mul(round, round, kpi, mask);
75             MicroAPI::Add2(round, round, (float)1.0, mask);
76             MicroAPI::Mul(round, round, xTmp, mask);
77             MicroAPI::Mul(dstReg, round, dstReg, mask);
78             MicroAPI::Mul(dstReg, dstReg, (float)1.0, mask);
79             MicroAPI::Max(dstReg, dstReg, (float)1.0, mask);
80             MicroAPI::DataCopy(dst + i * GetVecLen() / sizeof(float), dstReg, mask);
81         }
82     }
83 }
```



2-3个循环体并行执行



3个循环体并行执行



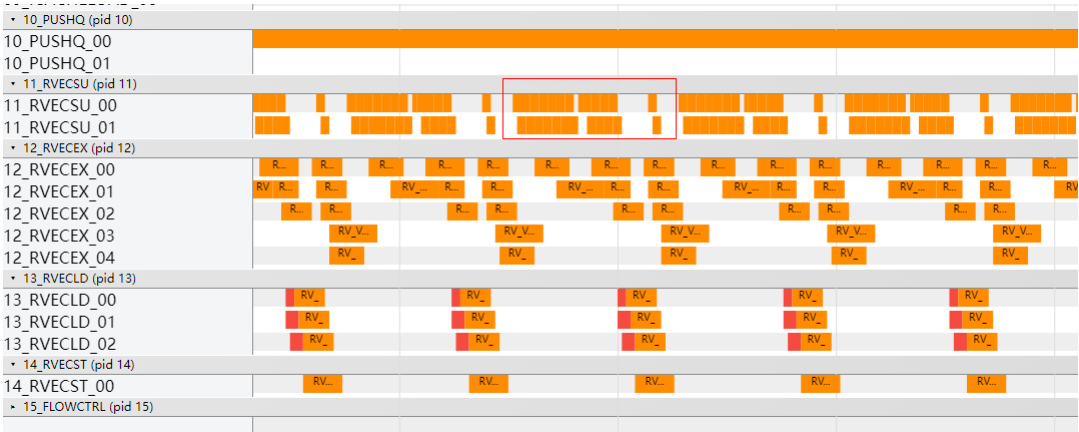
循环体串行执行

其他措施：

1.使用融合指令，减少指令总数

指令	融合公式
MulAddDst	$src0 * src1 + dst$
FusedMulDstAdd	$dst * src0 + src1$
FusedExpSub	$e^{(src0 - src1)}$
FusedAbsSub	$abs(src0 - src1)$

2.避免复杂重复Scalar计算阻塞流水



目录

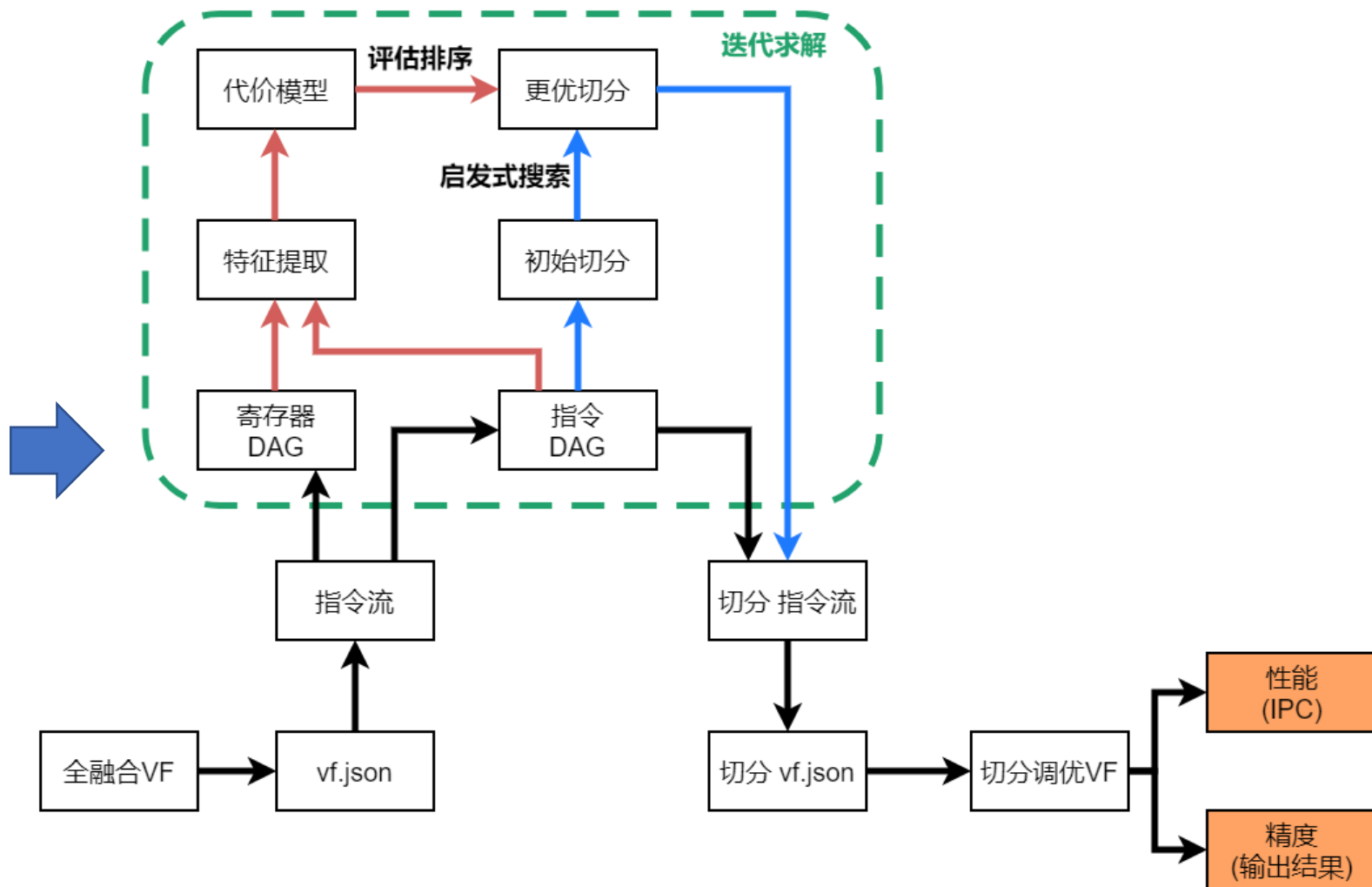
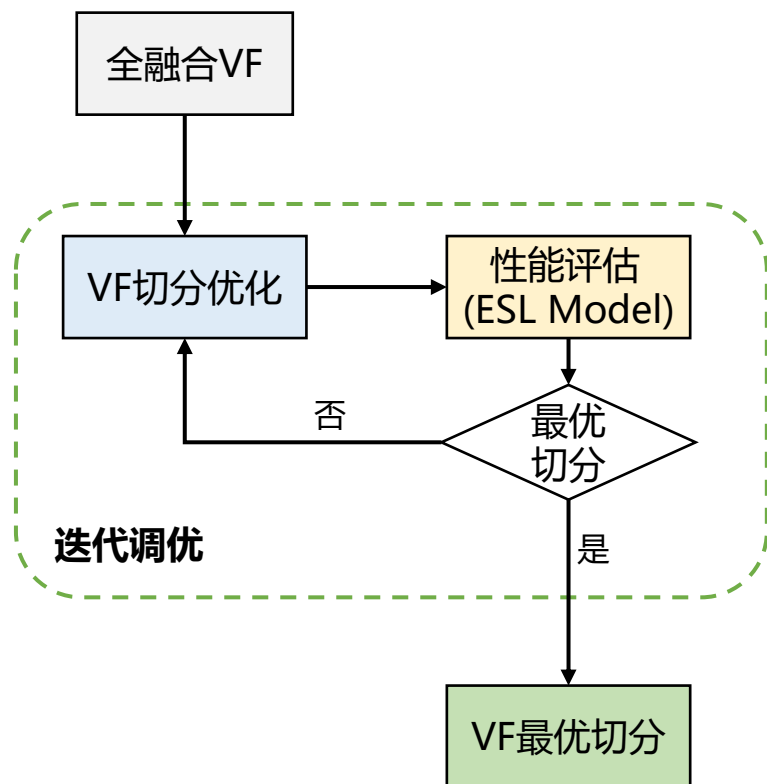
Part 1 Regbase编程回顾

Part 2 Regbase编程高性能实践

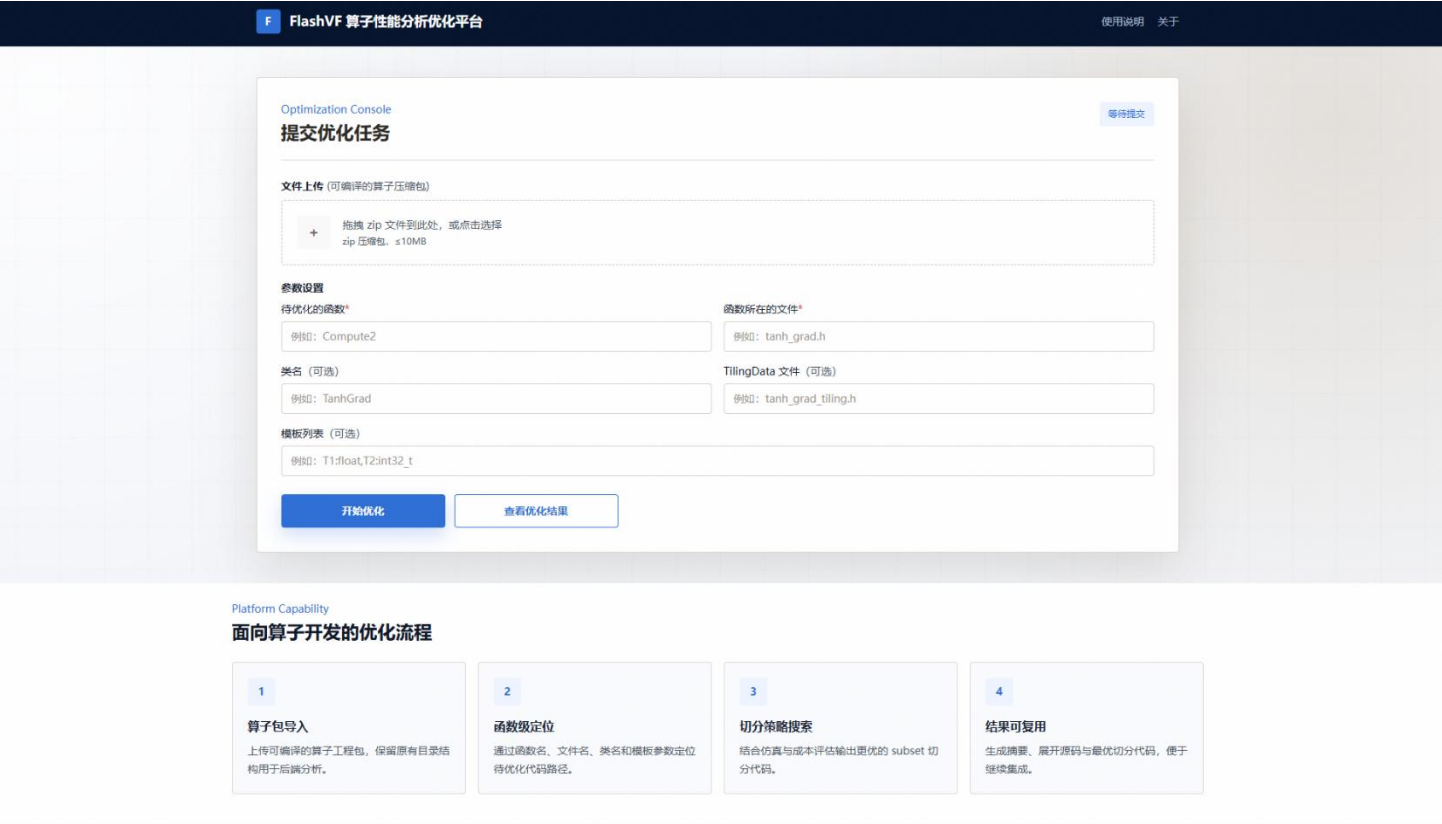
Part 3 基于FlashVF的性能优化实践

FlashVF工具介绍

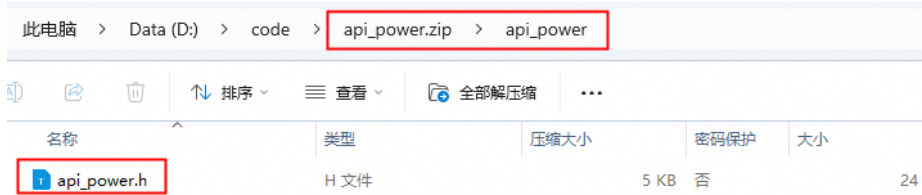
针对高复杂计算场景，模拟开发者调优过程，进行自动化调优



FlashVF工具演示



上传的可独立编译的算子压缩包结构如下：



参数设置：

1. 待优化的Regbase函数以及函数所在文件是必选项
2. 如果是类的成员函数，则需要设置类名
3. 如果有TilingData，则需要设置TilingData所在的文件
4. 模板列表填函数模板，如果有类模板也填入，格式如下：T:float,U:in32_tt,C:int64_t

FlashVF工具演示

Result Workspace

优化结果

已生成结果文件，可复制代码继续集成

优化结果文件

类型	代码	性能优化图
原始函数展开	partition_subset1_seed0.h	partition_subset1_seed0_vf_trace.json
最优切分Subset2	partition_subset2_seed142.h	partition_subset2_seed142_vf_trace.json
最优切分Subset3	partition_subset3_seed42.h	partition_subset3_seed42_vf_trace.json

下载结果

性能优化摘要

类型	CYCLE	IPC	SUBSET_NUM	BUFFER_NUM
原始函数展开	4945	1.11	1	0
partition_subset2_seed7842	4465	1.23	2	11
partition_subset3_seed9742	4196	1.33	3	11

源代码展开

partition_subset1_seed0.h

```
AscendC::Reg::Duplicate(tmpFloatReg_4, global_LOG2_REDUCE_COEFF1, mask_1);
AscendC::Reg::Sub(tmpEReg_4, (RegTensor<int32_t> &)tmpDstReg_1, (RegTensor<int32_t> &)tmpFloatReg_4, mask_1);
AscendC::Reg::Duplicate(tmpIntReg_4, global_LOG2_REDUCE_COEFF2, mask_1);
AscendC::Reg::And(tmpEReg_4, tmpEReg_4, tmpIntReg_4, mask_1);
AscendC::Reg::Sub((RegTensor<int32_t> &)tmpMReg_4, (RegTensor<int32_t> &)tmpDstReg_1, tmpEReg_4, mask_1);
AscendC::Reg::Cast<float, int32_t, defaultCastTrait>(tmpFloatReg_4, tmpEReg_4, mask_1);
AscendC::Reg::Duplicate(tmpFloatReg2_4, global_LOG2_REDUCE_FMAF_COEFF1, mask_1);
AscendC::Reg::MulAddDst(tmpIReg_4, tmpFloatReg_4, tmpFloatReg2_4, mask_1);
AscendC::Reg::Adds(tmpPReg_4, tmpMReg_4, temp_scalar_1_4, mask_1);
AscendC::Reg::Adds(tmpMReg_4, tmpMReg_4, temp_scalar_2_4, mask_1);
AscendC::Reg::Duplicate(tmpFloatReg_4, temp_scalar_3_4, mask_1);
AscendC::Reg::Div(tmpPReg_4, tmpFloatReg_4, tmpPReg_4, mask_1);
AscendC::Reg::Mul(tmpQHIREg_4, tmpMReg_4, tmpPReg_4, mask_1);
AscendC::Reg::Duplicate(tmpReg_5, temp_scalar_4_4, mask_1);
```

复制代码

- 优化结果说明：
1. 优化结果支持下载，包含优化后的函数代码以及性能优化图
 2. 性能优化图通过chrome浏览器的chrome://tracing插件查看
 3. 性能优化摘要描述了代码优化的指标，其中IPC越大，说明每秒指令数越多，则优化效果越好

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN